

Restaurant Management System (RMS) — Complete Technical Blueprint & Implementation Plan

Stack: Django + DRF • PostgreSQL • Docker/Compose • Bootstrap 5 templates (admin/cashier/KDS) • Flutter (customer app) • Paymob (payments for Egypt)

Author context: Solo software engineer in Zagazig, Sharqia, Egypt.

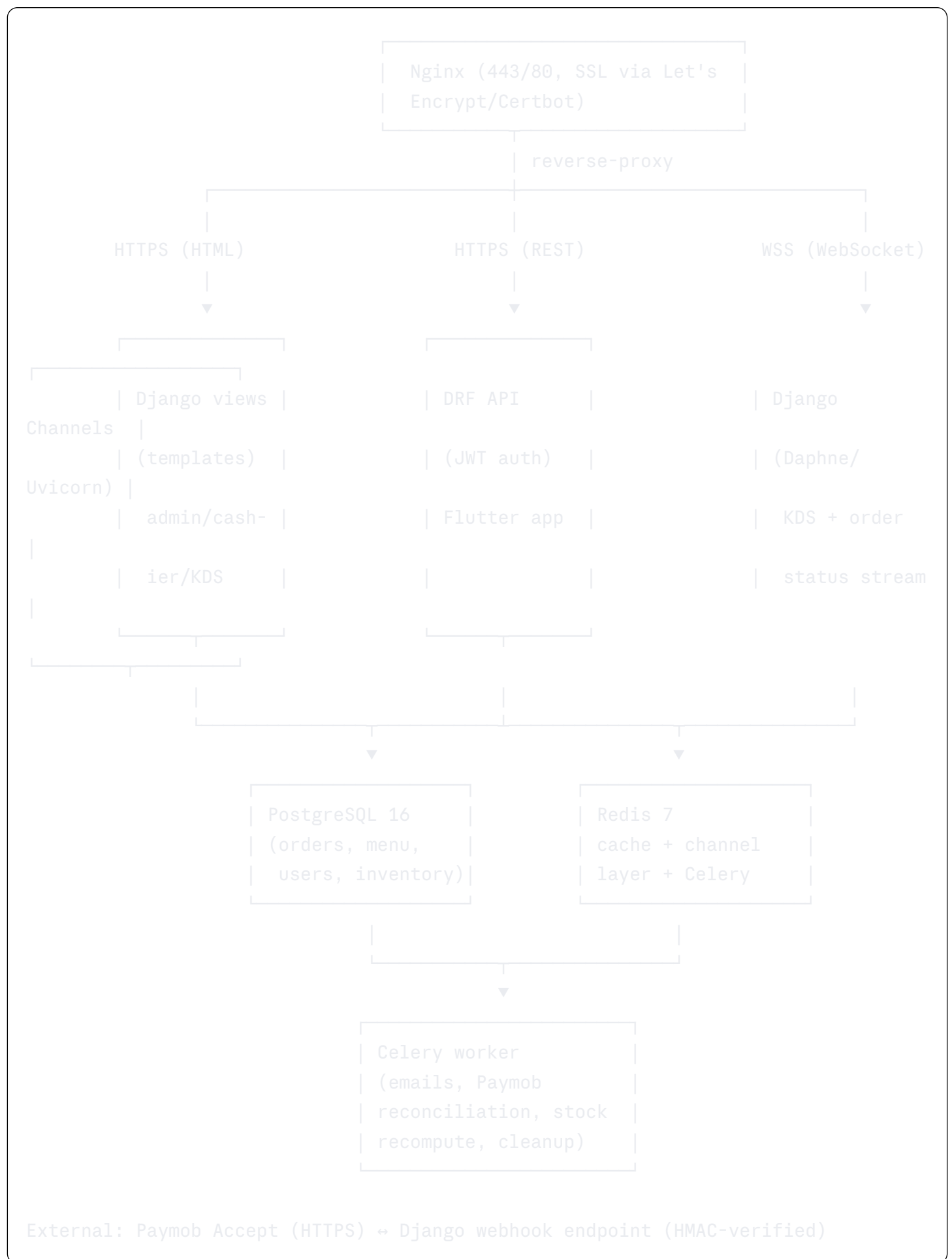
Goal: Production-oriented, security-first, built for the Egyptian market.

Executive Summary & Key Decisions

Decision	Choice	Why
Payment gateway	Paymob (Unified Checkout / Intention API)	PCI DSS Level 1, 3DS 2, native Egypt support, EGP settlement T+1, first-party Flutter SDK, Python integration samples, BNPL/wallet/Meeza support, 2.75% + 3 EGP pricing (Paymob Pricing ; bilixe Paymob review ; ThruHQ top 10)
Real-time KDS	Django Channels + Redis channel layer (WebSockets)	Native Django, scales via Redis, bi-directional updates for order status (TestDriven Channels ; OneUptime Channels guide)
Customer app auth	JWT (djangorestframework-simplejwt) with refresh rotation + blacklist	Stateless, mobile-friendly, (Django REST framework) officially documented in DRF (DRF auth docs)
Admin/cashier auth	Session auth (Django default)	Cookie+CSRF protection, SSR templates
Flutter state mgmt	Riverpod 3 (primary) with Bloc as acceptable alternative	Compile-time safety, lowest boilerplate, modern default for 2026 (Foresight Mobile 2026 ; Creole Studios)
Admin UI base	CoreUI v5 (Bootstrap 5, MIT) OR Tabler — customized theme	Both are Bootstrap-5 native, actively maintained, MIT (CoreUI ; AdminLTE review)
DB	PostgreSQL 16 with JSONB for order snapshots	Native JSONB + GIN indexing (pganalyze JSONB)
Caching / queue / channels	Redis 7	Single infrastructure for cache + Celery broker + channel layer

1. System Architecture

1.1 High-level component diagram (logical)



1.2 Dual-role Django: REST + Templates

Django serves two concurrent interfaces from a single codebase:

- `rms/urls.py` routes `/api/v1/*` to DRF `ViewSet`s (JSON, JWT-auth, CORS-enabled for the mobile app).
- `rms/urls.py` routes `/admin/`, `/cashier/`, `/kds/`, `/dashboard/` to classical Django views rendering Bootstrap 5 templates (session-auth, CSRF-protected).
- Business logic lives in **services/managers** (not in views) so both layers share the same code paths (e.g., `orders.services.create_order(cart, user)` is called by both the DRF serializer and the cashier HTML form).

1.3 Real-time KDS — recommendation

Use Django Channels with WebSockets (not polling, not SSE). Polling overwhelms the DB and introduces kitchen latency; SSE is one-way (fine for broadcasting but you want the kitchen to push "READY" back). Channels + Redis channel layer is the idiomatic Django approach and scales horizontally by adding more Daphne workers behind Nginx ([Django Channels + Redis pattern](#); [TestDriven tutorial](#)).

Concretely:

- ASGI server: **Daphne** (or Uvicorn) running alongside Gunicorn (Gunicorn handles sync HTTP, Daphne handles `/ws/`).
- Channel layer backend: `channels_redis`.
- Groups: `kds_station_<station_id>`, `order_<order_id>` (so customers subscribe to their own order).

1.4 Authentication strategy

Consumer	Scheme	Library	Storage
Flutter customer app	JWT (access 15 min + refresh 7 days, rotation + blacklist)	<code>djangoRESTframework-simplejwt</code>	<code>flutter_secure_storage</code> (Keystore/Keychain)
Admin web dashboard	Django session + CSRF	built-in <code>django.contrib.auth</code>	Signed HttpOnly cookie
Cashier web UI	Django session	built-in	Cookie
KDS websocket	Session cookie OR short-lived JWT passed as first WS message	<code>channels.auth.AuthMiddlewareStack</code>	—

`simplejwt` with `ROTATE_REFRESH_TOKENS=True` and `BLACKLIST_AFTER_ROTATION=True` is the recommended pattern for mobile clients (`Django REST framework`) ([DRF auth guide](#)).

2. Complete Django Data Models

Project layout:

```

backend/
├─ rms/                # project (settings, urls, asgi, wsgi, celery)
├─ accounts/           # User, Shift, Role
├─ menu/               # Category, MenuItem, Modifier, ModifierOption
├─ inventory/          # Ingredient, Recipe, StockTransaction
├─ orders/              # Table, Order, OrderItem, OrderItemModifier
├─ payments/           # Payment, PaymobIntention
├─ loyalty/             # LoyaltyTier, CustomerPoints, PointsLedger
└─ analytics/          # materialized views / cached reports

```

2.1 `accounts/models.py`

python

```
from django.contrib.auth.models import AbstractUser
from django.db import models
from django.utils.translation import gettext_lazy as _
```

```
class Role(models.TextChoices):
    ADMIN      = "admin",      _("Administrator")
    MANAGER    = "manager",    _("Manager")
    CASHIER     = "cashier",    _("Cashier")
    KITCHEN    = "kitchen",    _("Kitchen staff")
    WAITER     = "waiter",     _("Waiter")
    CUSTOMER   = "customer",   _("Customer")
```

```
class User(AbstractUser):
    # email as primary login (keep username for admin compat)
    email      = models.EmailField(_("email address"), unique=True)
    phone      = models.CharField(max_length=20, blank=True, db_index=True)
    role       = models.CharField(max_length=16, choices=Role.choices, default=Role.ADMIN)
    is_active  = models.BooleanField(default=True)
    date_joined = models.DateTimeField(auto_now_add=True)

    # profile
    avatar     = models.ImageField(upload_to="avatars/", blank=True, null=True)
    address    = models.TextField(blank=True)

    # loyalty cached counter (source of truth is PointsLedger)
    loyalty_points = models.PositiveIntegerField(default=0)

    USERNAME_FIELD = "email"
    REQUIRED_FIELDS = ["username"]
```

```
class Meta:
    indexes = [models.Index(fields=["role", "is_active"])]
```

```
@property
def is_staff_member(self):
    return self.role in {Role.ADMIN, Role.MANAGER, Role.CASHIER, Role.KITCHEN, Role.WAITER}
```

```
class Shift(models.Model):
    user      = models.ForeignKey(User, on_delete=models.CASCADE, related_name="shifts")
    started_at = models.DateTimeField()
    ended_at  = models.DateTimeField(null=True, blank=True)
    opening_cash = models.DecimalField(max_digits=12, decimal_places=2, default=0)
    closing_cash = models.DecimalField(max_digits=12, decimal_places=2, null=True, blank=True)
    notes      = models.TextField(blank=True)
```

```
class Meta:
    indexes = [models.Index(fields=["user", "started_at"])]
```

```
indexes = models.Index(fields=["user", "started_at"])]
```

Subclassing `AbstractUser` is the recommended pattern over `AbstractBaseUser` for most projects [\(LearnDjango\)](#) ([LearnDjango custom user](#); [TestDriven custom user](#)). Set `AUTH_USER_MODEL = "accounts.User"` in settings **before the first migration**.

2.2 [menu/models.py](#)

python

```
from django.db import models
from django.core.validators import MinValueValidator

class Category(models.Model):
    name = models.CharField(max_length=80, unique=True)
    slug = models.SlugField(unique=True)
    description = models.TextField(blank=True)
    image = models.ImageField(upload_to="categories/", blank=True, null=True)
    display_order = models.PositiveSmallIntegerField(default=0)
    is_active = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["display_order", "name"]

class MenuItem(models.Model):
    class Type(models.TextChoices):
        DISH = "dish", "Dish"
        DRINK = "drink", "Drink"
        COMBO = "combo", "Combo"

    category = models.ForeignKey(Category, on_delete=models.PROTECT, related_name='items')
    name = models.CharField(max_length=120)
    name_ar = models.CharField(max_length=120, blank=True) # Arabic for EG
    description = models.TextField(blank=True)
    item_type = models.CharField(max_length=10, choices=Type.choices, default=Type.DISH)
    price = models.DecimalField(max_digits=10, decimal_places=2, validators=[MinValueValidator(0)])
    cost = models.DecimalField(max_digits=10, decimal_places=2, default=0, validators=[MinValueValidator(0)])
    image = models.ImageField(upload_to="menu/", blank=True, null=True)
    is_available = models.BooleanField(default=True, db_index=True) # quick toggle
    prep_time_min = models.PositiveSmallIntegerField(default=10)
    calories = models.PositiveIntegerField(null=True, blank=True)
    tags = models.JSONField(default=list, blank=True) # ["spicy", "vegan"]
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        indexes = [
            models.Index(fields=["category", "is_available"]),
            models.Index(fields=["item_type", "is_available"]),
        ]
        ordering = ["category__display_order", "name"]

    def __str__(self):
        return self.name
```

```
class ModifierGroup(models.Model):
    """e.g. 'Size', 'Extra toppings'."""
    menu_item = models.ForeignKey(MenuItem, on_delete=models.CASCADE, related_name=
    name = models.CharField(max_length=80)
    min_select = models.PositiveSmallIntegerField(default=0)
    max_select = models.PositiveSmallIntegerField(default=1)
    is_required = models.BooleanField(default=False)

class ModifierOption(models.Model):
    group = models.ForeignKey(ModifierGroup, on_delete=models.CASCADE, related
    name = models.CharField(max_length=80)
    price_delta = models.DecimalField(max_digits=8, decimal_places=2, default=0)
    is_available= models.BooleanField(default=True)
```

2.3 `inventory/models.py`

python

```
from django.db import models
```

```
class Ingredient(models.Model):
```

```
    class Unit(models.TextChoices):
```

```
        GRAM = "g", "Grams"
```

```
        KG = "kg", "Kilograms"
```

```
        ML = "ml", "Milliliters"
```

```
        L = "l", "Liters"
```

```
        PIECE = "pc", "Piece"
```

```
    name = models.CharField(max_length=100, unique=True)
```

```
    unit = models.CharField(max_length=4, choices=Unit.choices)
```

```
    stock_qty = models.DecimalField(max_digits=12, decimal_places=3, default=0)
```

```
    low_threshold = models.DecimalField(max_digits=12, decimal_places=3, default=0)
```

```
    cost_per_unit = models.DecimalField(max_digits=10, decimal_places=4, default=0)
```

```
    supplier = models.CharField(max_length=120, blank=True)
```

```
    updated_at = models.DateTimeField(auto_now=True)
```

```
    @property
```

```
    def is_low(self):
```

```
        return self.stock_qty <= self.low_threshold
```

```
class Recipe(models.Model):
```

```
    """M2M link MenuItem <-> Ingredient with qty used per one sold item."""
```

```
    menu_item = models.ForeignKey("menu.MenuItem", on_delete=models.CASCADE, related_name="recipes")
```

```
    ingredient = models.ForeignKey(Ingredient, on_delete=models.PROTECT, related_name="recipes")
```

```
    quantity = models.DecimalField(max_digits=10, decimal_places=3)
```

```
    class Meta:
```

```
        unique_together = [("menu_item", "ingredient")]
```

```
class StockTransaction(models.Model):
```

```
    class Kind(models.TextChoices):
```

```
        PURCHASE = "purchase", "Purchase"
```

```
        SALE = "sale", "Sale deduction"
```

```
        WASTE = "waste", "Waste / spoilage"
```

```
        ADJUST = "adjust", "Manual adjustment"
```

```
    ingredient = models.ForeignKey(Ingredient, on_delete=models.PROTECT, related_name="stock_transactions")
```

```
    kind = models.CharField(max_length=10, choices=Kind.choices)
```

```
    quantity = models.DecimalField(max_digits=12, decimal_places=3, help_text="positive or negative")
```

```
    unit_cost = models.DecimalField(max_digits=10, decimal_places=4, default=0)
```

```
    order = models.ForeignKey("orders.Order", null=True, blank=True, on_delete=models.SET_NULL)
```

```
    created_by = models.ForeignKey("accounts.User", null=True, on_delete=models.SET_NULL)
```

```
created_at = models.DateTimeField(auto_now_add=True, db_index=True)
note = models.TextField(blank=True)
```

```
class Meta:
    indexes = [models.Index(fields=["ingredient", "created_at"])]
```

2.4 orders/models.py

python

```
import uuid
from django.db import models
from django.conf import settings

class Table(models.Model):
    number = models.PositiveSmallIntegerField(unique=True)
    capacity = models.PositiveSmallIntegerField(default=4)
    qr_token = models.UUIDField(default=uuid.uuid4, unique=True) # scan-to-order
    is_active = models.BooleanField(default=True)

class Order(models.Model):
    class Type(models.TextChoices):
        DINE_IN = "dine_in", "Dine-in"
        TAKEAWAY = "takeaway", "Takeaway"
        DELIVERY = "delivery", "Delivery"

    class Status(models.TextChoices):
        DRAFT = "draft", "Draft"
        PENDING = "pending", "Pending payment"
        CONFIRMED = "confirmed", "Confirmed / in kitchen"
        PREPARING = "preparing", "Preparing"
        READY = "ready", "Ready"
        SERVED = "served", "Served / delivered"
        CANCELLED = "cancelled", "Cancelled"
        REFUNDED = "refunded", "Refunded"

    class PaymentStatus(models.TextChoices):
        UNPAID = "unpaid", "Unpaid"
        PAID = "paid", "Paid"
        FAILED = "failed", "Failed"
        REFUNDED = "refunded", "Refunded"

    uuid = models.UUIDField(default=uuid.uuid4, unique=True, editable=False)
    number = models.CharField(max_length=12, unique=True, db_index=True) #
    customer = models.ForeignKey(settings.AUTH_USER_MODEL, null=True, blank=True, on_delete=models.SET_NULL, related_name="orders")
    cashier = models.ForeignKey(settings.AUTH_USER_MODEL, null=True, blank=True, on_delete=models.SET_NULL, related_name="handled_orders")
    table = models.ForeignKey(Table, null=True, blank=True, on_delete=models.SET_NULL)
    order_type = models.CharField(max_length=10, choices=Type.choices, default=Type.DINE_IN)
    status = models.CharField(max_length=12, choices=Status.choices, default=Status.DRAFT)
    payment_status = models.CharField(max_length=10, choices=PaymentStatus.choices, default=PaymentStatus.UNPAID, db_index=True)

    subtotal = models.DecimalField(max_digits=12, decimal_places=2, default=0)
```

```

tax = models.DecimalField(max_digits=12, decimal_places=2, default=0)
service_charge = models.DecimalField(max_digits=12, decimal_places=2, default=0)
discount = models.DecimalField(max_digits=12, decimal_places=2, default=0)
total = models.DecimalField(max_digits=12, decimal_places=2, default=0)

delivery_address = models.TextField(blank=True)
notes = models.TextField(blank=True)
# frozen snapshot of item prices/modifiers for audit – JSONB on Postgres
snapshot = models.JSONField(default=dict, blank=True)

created_at = models.DateTimeField(auto_now_add=True, db_index=True)
confirmed_at = models.DateTimeField(null=True, blank=True)
ready_at = models.DateTimeField(null=True, blank=True)
served_at = models.DateTimeField(null=True, blank=True)

class Meta:
    indexes = [
        models.Index(fields=["created_at"]),
        models.Index(fields=["status", "created_at"]),
        models.Index(fields=["customer", "-created_at"]),
        models.Index(fields=["payment_status", "status"]),
    ]
    ordering = ["-created_at"]

def recalc_totals(self):
    self.subtotal = sum(i.line_total for i in self.items.all())
    self.tax = self.subtotal * 0 # configure VAT rate per country
    self.total = self.subtotal + self.tax + self.service_charge - self.discount

class OrderItem(models.Model):
    order = models.ForeignKey(Order, on_delete=models.CASCADE, related_name="items")
    menu_item = models.ForeignKey("menu.MenuItem", on_delete=models.PROTECT)
    quantity = models.PositiveSmallIntegerField(default=1)
    unit_price = models.DecimalField(max_digits=10, decimal_places=2) # frozen at
    notes = models.TextField(blank=True)

    @property
    def modifiers_total(self):
        return sum(m.price_delta for m in self.chosen_modifiers.all())

    @property
    def line_total(self):
        return (self.unit_price + self.modifiers_total) * self.quantity

class OrderItemModifier(models.Model):
    order_item = models.ForeignKey(OrderItem, on_delete=models.CASCADE, related_name="modifiers")
    option = models.ForeignKey("menu.ModifierOption", on_delete=models.PROTECT)
    price_delta = models.DecimalField(max_digits=8, decimal_places=2) # frozen

```

2.5 payments/models.py

python

```
from django.db import models
```

```
class Payment(models.Model):
```

```
    class Method(models.TextChoices):
```

```
        CASH = "cash", "Cash"
        CARD = "card", "Visa/Mastercard (Paymob)"
        WALLET = "wallet", "Mobile wallet (Paymob)"
        KIOSK = "kiosk", "Paymob kiosk / Aman"
        BNPL = "bnpl", "ValU / Halan / Symp1"
```

```
    class Status(models.TextChoices):
```

```
        INITIATED = "initiated", "Initiated"
        PENDING = "pending", "Pending 3DS"
        SUCCESS = "success", "Success"
        FAILED = "failed", "Failed"
        REFUNDED = "refunded", "Refunded"
```

```
    order = models.ForeignKey("orders.Order", on_delete=models.CASCADE, related_name="payments")
    method = models.CharField(max_length=12, choices=Method.choices)
    amount = models.DecimalField(max_digits=12, decimal_places=2)
    currency = models.CharField(max_length=3, default="EGP")
    status = models.CharField(max_length=10, choices=Status.choices, default="initiated")
    gateway = models.CharField(max_length=20, default="paymob")
    gateway_txn_id = models.CharField(max_length=80, blank=True, db_index=True)
    raw_response = models.JSONField(default=dict, blank=True) # auditing
    created_at = models.DateTimeField(auto_now_add=True, db_index=True)
```

```
class PaymobIntention(models.Model):
```

```
    """Tracks Paymob intention state separately from Payment (1-to-many: a failed intention
    can retry and spawn a new Payment record)."""
```

```
    order = models.ForeignKey("orders.Order", on_delete=models.CASCADE, related_name="intentions")
    client_secret = models.CharField(max_length=255, db_index=True)
    intention_id = models.CharField(max_length=80, unique=True)
    amount_cents = models.BigIntegerField()
    special_reference = models.CharField(max_length=80, unique=True)
    is_confirmed = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)
```

2.6 loyalty/models.py

python

```
from django.db import models
from django.conf import settings

class LoyaltyTier(models.Model):
    name = models.CharField(max_length=50, unique=True)
    min_points = models.PositiveIntegerField()
    earn_multiplier = models.DecimalField(max_digits=4, decimal_places=2, default=1)

class PointsLedger(models.Model):
    class Kind(models.TextChoices):
        EARN = "earn", "Earn"
        REDEEM = "redeem", "Redeem"
        EXPIRE = "expire", "Expired"
        ADJUST = "adjust", "Manual adjustment"

    user = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE,
                             order = models.ForeignKey("orders.Order", null=True, blank=True, on_delete=models.CASCADE))
    order = models.ForeignKey("orders.Order", null=True, blank=True, on_delete=models.CASCADE)
    kind = models.CharField(max_length=10, choices=Kind.choices)
    points = models.IntegerField(help_text="positive=credit, negative=debit")
    balance_after = models.IntegerField()
    created_at = models.DateTimeField(auto_now_add=True, db_index=True)

    class Meta:
        indexes = [models.Index(fields=["user", "-created_at"])]
```

3. PostgreSQL Schema Considerations

3.1 Indexing strategy

- **Time-series queries (most common):** B-tree on `(orders.created_at)`, composite `((status, created_at))`, `((customer, -created_at))`.
- **Menu browsing:** `((category_id, is_available))`, `((item_type, is_available))`.
- **Gateway reconciliation:** unique index on `(payments.gateway_txn_id)` (already via `db_index=True` + uniqueness at app level).
- **Full-text search on menu names (optional):** `GIN(to_tsvector('simple', name || ' ' || description))` via `(django.contrib.postgres.indexes.GinIndex)`.
- **Tags JSONB search:** `GIN` index on `(MenuItem.tags)` using `(jsonb_path_ops)` for containment queries; supports `(WHERE tags @> '["vegan"]')` in milliseconds even at millions of rows (`(pganalyze GIN)`).

Example (add to `menu/models.py`):

```
python

from django.contrib.postgres.indexes import GinIndex

class Meta:
    indexes = [
        models.Index(fields=["category", "is_available"]),
        GinIndex(fields=["tags"], name="menuitem_tags_gin"),
    ]
```

3.2 JSONB fields to use

- `Order.snapshot` — freeze a copy of the menu item names/prices/modifiers at checkout so that even if you delete/rename a dish later, the order invoice remains faithful. JSONB is the preferred choice because it stores data in a decomposed binary format enabling efficient indexing ([pganalyze JSONB](#)).
- `MenuItem.tags` — flexible list of tags (`["vegan", "spicy", "new"]`).
- `Payment.raw_response` — opaque gateway payload for audit.

3.3 Database constraints & integrity

- `on_delete=PROTECT` on `OrderItem.menu_item` and `Recipe.ingredient` — prevents accidental deletion of things in active orders; business-critical.
- `on_delete=SET_NULL` on `Order.customer`/`cashier` — preserves historical orders if a user is removed.
- `CheckConstraint` on `OrderItem.quantity > 0`, `Payment.amount >= 0`:

```
python

class Meta:
    constraints = [
        models.CheckConstraint(check=models.Q(quantity__gt=0), name="qty_positive"),
    ]
```

- Wrap stock deduction + order confirmation in `transaction.atomic()` with `select_for_update()` on `Ingredient` rows to avoid oversell races.

4. REST API Endpoints for Flutter App

All endpoints prefixed `/api/v1/`. JSON requests/responses. JWT bearer required unless noted.

4.1 Authentication

Method	Path	Auth	Purpose
POST	<code>/auth/register/</code>	public	Create customer; body: <code>{email, password, name, phone}</code>
POST	<code>/auth/login/</code>	public	Returns <code>{access, refresh, user}</code>
POST	<code>/auth/refresh/</code>	refresh token	Rotate tokens
POST	<code>/auth/logout/</code>	access	Blacklists refresh token
POST	<code>/auth/password/change/</code>	access	—
POST	<code>/auth/password/reset/</code>	public	Email OTP

4.2 Menu

Method	Path	Purpose
GET	<code>/menu/categories/</code>	List active categories
GET	<code>/menu/items/</code>	<code>?category=slug&search=&type=dish&available=true&ordering=price</code>
GET	<code>/menu/items/{id}/</code>	Item detail + modifier groups & options
GET	<code>/menu/featured/</code>	Homepage carousel (cached 60s)

4.3 Cart & Orders

Method	Path	Purpose
POST	<code>/orders/</code>	Create order (type, items, modifiers, notes, delivery_address)
GET	<code>/orders/{uuid}/</code>	Order detail
GET	<code>/orders/</code>	<code>?status=&page=</code> customer's history
PATCH	<code>/orders/{uuid}/cancel/</code>	Customer cancels (if status in {draft,pending})
GET	<code>/orders/{uuid}/status/</code>	Short poll fallback (primary is WS)

4.4 Payments

Method	Path	Purpose
POST	/payments/ intention/	Body {order_uuid, method} → returns {client_secret, public_key, checkout_url}
POST	/payments/webhook/ paymob/	public, HMAC-verified. Paymob callback → mark order paid
GET	/payments/{id}/	Verify status

4.5 Customer profile & loyalty

Method	Path	Purpose
GET	/me/	Current user profile
PATCH	/me/	Update name/phone/avatar/address
GET	/me/loyalty/	{balance, tier, history}
GET	/me/addresses/	Saved delivery addresses

4.6 KDS (WebSocket)

- wss://host/ws/kds/?token=<JWT> (for kitchen staff)
- wss://host/ws/order/<uuid>/ (for customer-facing live order tracker)

Messages:

```
json

// server → kitchen
{"event": "order.new", "order": {...}}
{"event": "order.updated", "order_id": 123, "status": "preparing"}
// kitchen → server
{"event": "order.status", "order_id": 123, "status": "ready"}
```

5. Payment Integration — Deep Comparison for Egypt

5.1 Comparison matrix

Gateway	EG availability	Methods	PCI / 3DS	Python/ Django	Flutter SDK	Fees (EG)
Paymob	✅ local, market leader	Visa, MC, Meeza, mobile wallets, Fawry kiosk, ValU/ Halan/ Sympl BNPL (bilixe)	PCI DSS Level 1, 3DS2, tokenization (ThruHQ)	Community Python libs + REST (GitHub Paymob ; sample Django integration)	Official Flutter SDK + multiple community packages (pub.dev paymob ; flutter_paymob)	2.75% + 3 EGP per transaction, no monthly (Paymob pricing ; inai.io)
Kashier	✅ local	Cards, wallets; fewer BNPL	PCI-DSS, 3DS	REST only (no official Python SDK)	No first-party Flutter SDK (Kashier developers)	Comparable on request
Fawry Pay	✅ local, unmatched offline network	Cards, reference-at-POS, wallet — biggest offline reach (Fawry dev)	PCI, 3DS	Community only (Ruby gem ; no mature Python)	No first-party Flutter SDK	Varies
MyFatoorah	Regional (GCC-first), Egypt supported	Cards, KNET, Apple Pay, Fawry	PCI, 3DS	REST	Community SDK	Higher vs local
Stripe	❌ Not supported in Egypt (Rapyd ; OneSafe ; doola) — requires US LLC + US bank	Global	Gold standard	Excellent	Excellent	2.9%+ higher for MENA
PayTabs	✅	Cards, wallets	PCI, 3DS	REST	Community	2.75% + 3 EGP (road9media)
Amazon	✅	Cards,	PCI Level 1,	REST +	Community	On request

Payment	wallets,	3DS2	SDKs
Services	installments		
(ex-			
PayFort)			

5.2 Winner: Paymob

Paymob is the recommended choice for an Egypt-based SME restaurant because:

1. **Widest local method coverage** — Visa/MC/Meeza + mobile wallets + Fawry kiosk + ValU/Halan BNPL from one integration; critical because a significant percentage of Egyptian e-commerce customers still prefer kiosk/BNPL over cards ([bilixe Paymob](#)).
2. **PCI DSS Level 1 certification** with native 3DS 2.0 on all card transactions — removes all PCI scope from your Django backend ([ThruHQ 2025](#)).
3. **First-party Flutter SDK** ([pub.dev paymob](#)) plus the official `PaymobAccept/flutter_sdk` and `paymob-python` GitHub repositories maintained by Paymob themselves ([Paymob GitHub org](#)).
4. **Simple Unified Checkout (Intention API)** — a single backend call produces a `client_secret` you drop into the SDK; no card data ever touches your server ([Paymob Intention API docs](#); [Paymob Intention endpoint](#)).
5. **Transparent pricing** (2.75% + 3 EGP, no monthly fees) and **T+1 settlement** to an Egyptian bank account ([Paymob pricing](#); [bilixe](#)).
6. **HMAC-signed webhooks** with a well-documented verification procedure — essential for reliable reconciliation ([Paymob HMAC docs referenced in sample code](#)).

5.3 Step-by-step Paymob integration

5.3.1 Dashboard setup (one-off)

1. Register at `accept.paymob.com`, complete KYC (commercial register, tax card, owner ID, bank account) ([road9media](#)).
2. In **Settings** → **Account Info** copy `API Key`, `Secret Key`, `Public Key`, `HMAC` ([Odoo guide to Paymob](#)).
3. In **Developers** → **Payment Integrations**, note the **integration IDs** for each method (card, wallet, kiosk) — each one must match your account mode (live/test) ([Paymob Intention API errors](#)).
4. Register your webhook URL: `https://yourdomain.com/api/v1/payments/webhook/paymob/`.

5.3.2 Django backend (Intention API flow)

python

```
# payments/services.py
import hmac, hashlib, requests
from django.conf import settings

PAYMOB_BASE = "https://accept.paymob.com/v1/intention/"

def create_intention(order) -> dict:
    items = [{
        "name": oi.menu_item.name[:50],
        "amount": int(oi.line_total * 100),    # piasters
        "description": oi.menu_item.name[:100],
        "quantity": oi.quantity,
    } for oi in order.items.all()]

    billing = {
        "first_name": order.customer.first_name or "Guest",
        "last_name": order.customer.last_name or "Customer",
        "email": order.customer.email,
        "phone_number": order.customer.phone or "+2010000000000",
        "country": "EGY",
        "city": "Cairo",
        "street": order.delivery_address[:60] or "NA",
        "building": "NA", "apartment": "NA", "floor": "NA", "state": "NA",
    }

    payload = {
        "amount": int(order.total * 100),
        "currency": "EGP",
        "payment_methods": settings.PAYMOB_INTEGRATION_IDS, # [CARD_ID, WALLET_ID,
        "items": items,
        "billing_data": billing,
        "special_reference": f"order-{order.uuid}",
        "notification_url": settings.PAYMOB_WEBHOOK_URL,
        "redirection_url": settings.PAYMOB_REDIRECT_URL,
    }

    r = requests.post(
        PAYMOB_BASE,
        json=payload,
        headers={"Authorization": f"Token {settings.PAYMOB_SECRET_KEY}"},
        timeout=15,
    )
    r.raise_for_status()
    data = r.json()
    PaymobIntention.objects.create(
        order=order,
        intention_id=data["id"],
        # ... other fields ...
    )
```

```

        client_secret=data["client_secret"],
        amount_cents=payload["amount"],
        special_reference=payload["special_reference"],
    )
    return {
        "client_secret": data["client_secret"],
        "public_key": settings.PAYMOB_PUBLIC_KEY,
        "checkout_url": f"https://accept.paymob.com/unifiedcheckout/?publicKey={set
    }

```

The response structure matches Paymob's documented Intention API ([Paymob Intention API reference](#); confirmed pattern in [sample Django+Paymob integration](#)).

5.3.3 Webhook (HMAC-verified)

Paymob callbacks are signed with HMAC-SHA512 over a specific ordered string of fields ([GitHub omaarsaaleh sample](#); [xdot-paymob .NET SDK which implements same algorithm](#)).

python

```
# payments/views.py
@api_view(["POST"])
@permission_classes([AllowAny])
def paymob_webhook(request):
    incoming_hmac = request.query_params.get("hmac")
    obj = request.data.get("obj", {})

    # Build the concatenated string Paymob expects (order of fields from their docs)
    parts = [
        obj["amount_cents"], obj["created_at"], obj["currency"],
        obj["error_occured"], obj["has_parent_transaction"], obj["id"],
        obj["integration_id"], obj["is_3d_secure"], obj["is_auth"],
        obj["is_capture"], obj["is_refunded"], obj["is_standalone_payment"],
        obj["is_voided"], obj["order"]["id"], obj["owner"], obj["pending"],
        obj["source_data"]["pan"], obj["source_data"]["sub_type"],
        obj["source_data"]["type"], obj["success"],
    ]
    concat = "".join(str(x).lower() if isinstance(x, bool) else str(x) for x in parts)
    expected = hmac.new(settings.PAYMOB_HMAC.encode(), concat.encode(), hashlib.sha512).hexdigest()

    if not hmac.compare_digest(expected, incoming_hmac):
        return Response(status=403)

    reference = obj["order"].get("merchant_order_id") # "order-<uuid>"
    try:
        order = Order.objects.get(uuid=reference.replace("order-", ""))
    except Order.DoesNotExist:
        return Response(status=404)

    if obj.get("success") and not obj.get("is_refunded"):
        with transaction.atomic():
            order.payment_status = Order.PaymentStatus.PAID
            order.status = Order.Status.CONFIRMED
            order.confirmed_at = timezone.now()
            order.save()
            Payment.objects.create(
                order=order,
                method=Payment.Method.CARD,
                amount=obj["amount_cents"] / 100,
                status=Payment.Status.SUCCESS,
                gateway_txn_id=str(obj["id"]),
                raw_response=obj,
            )
            # deduct inventory, emit KDS event, credit loyalty points
            orders.services.on_order_paid(order)
    else:
        Payment.objects.create(order=order, method=Payment.Method.CARD,
```

```
        amount=obj["amount_cents"] / 100,
        status=Payment.Status.FAILED,
        gateway_txn_id=str(obj["id"]),
        raw_response=obj)

    return Response(status=200)
```

5.3.4 Flutter side

Use the official `paymob` package (pub.dev/paymob) or `flutter_paymob` ([Flutter Gems entry](#)):

```
dart

// 1. Call backend to create intention
final res = await dio.post('/payments/intention/',
    data: {'order_uuid': orderUuid, 'method': 'card'});
final clientSecret = res.data['client_secret'];
final publicKey    = res.data['public_key'];

// 2. Open Unified Checkout via WebView or SDK
await FlutterPaymob.instance.initialize(
    publicKey: publicKey,
    clientSecret: clientSecret,
);
final result = await FlutterPaymob.instance.payWithCard(context: context);
if (result.success) {
    // Navigate to "tracking" screen; real status comes via WebSocket + webhook
}
```

The SDK natively handles 3DS redirect inside the WebView, so your app never sees the card PAN — keeping your PCI scope to SAQ-A only.

6. Docker & Deployment Setup

6.1 `Dockerfile` (multi-stage, production)

dockerfile

```
# ----- build stage -----  
FROM python:3.12-slim AS builder  
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1  
WORKDIR /app  
RUN apt-get update && apt-get install -y --no-install-recommends \  
    build-essential libpq-dev gcc && rm -rf /var/lib/apt/lists/*  
COPY requirements.txt .  
RUN pip wheel --no-cache-dir --wheel-dir /wheels -r requirements.txt  
  
# ----- runtime stage -----  
FROM python:3.12-slim  
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 \  
    DJANGO_SETTINGS_MODULE=rms.settings.production  
RUN apt-get update && apt-get install -y --no-install-recommends \  
    libpq5 curl && rm -rf /var/lib/apt/lists/*  
RUN addgroup --system app && adduser --system --group app  
WORKDIR /app  
COPY --from=builder /wheels /wheels  
COPY requirements.txt .  
RUN pip install --no-cache /wheels/*  
COPY --chown=app:app . .  
RUN mkdir -p /app/staticfiles /app/mediafiles && chown -R app:app /app  
USER app  
EXPOSE 8000  
CMD ["gunicorn", "rms.wsgi:application", \  
    "--bind", "0.0.0.0:8000", "--workers", "4", \  
    "--access-logfile", "-", "--error-logfile", "-"]
```

6.2 `docker-compose.yml` (production)

yaml

```
version: "3.9"
```

```
services:
```

```
  db:
```

```
    image: postgres:16-alpine
```

```
    restart: always
```

```
    volumes:
```

```
      - pgdata:/var/lib/postgresql/data
```

```
    environment:
```

```
      POSTGRES_DB: ${POSTGRES_DB}
```

```
      POSTGRES_USER: ${POSTGRES_USER}
```

```
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
```

```
    healthcheck:
```

```
      test: ["CMD-SHELL", "pg_isready -U $$POSTGRES_USER"]
```

```
      interval: 10s
```

```
      retries: 5
```

```
  redis:
```

```
    image: redis:7-alpine
```

```
    restart: always
```

```
    volumes:
```

```
      - redisdata:/data
```

```
  web:
```

```
    build: .
```

```
    restart: always
```

```
    env_file: .env
```

```
    depends_on:
```

```
      db:
```

```
        condition: service_healthy
```

```
      redis:
```

```
        condition: service_started
```

```
    volumes:
```

```
      - staticfiles:/app/staticfiles
```

```
      - mediafiles:/app/mediafiles
```

```
    command: >
```

```
      sh -c "python manage.py migrate --noinput &&
```

```
              python manage.py collectstatic --noinput &&
```

```
              gunicorn rms.wsgi:application --bind 0.0.0.0:8000 --workers 4"
```

```
  asgi:  # Daphne for Channels/WebSockets
```

```
    build: .
```

```
    restart: always
```

```
    env_file: .env
```

```
    depends_on: [db, redis]
```

```
    command: daphne -b 0.0.0.0 -p 9000 rms.asgi:application
```

```

celery:
  build: .
  restart: always
  env_file: .env
  depends_on: [db, redis]
  command: celery -A rms worker -l info --concurrency=2

celery-beat:
  build: .
  restart: always
  env_file: .env
  depends_on: [db, redis]
  command: celery -A rms beat -l info

nginx:
  image: nginx:1.27-alpine
  restart: always
  ports: ["80:80", "443:443"]
  volumes:
    - ./nginx/conf.d:/etc/nginx/conf.d:ro
    - staticfiles:/var/www/static:ro
    - mediafiles:/var/www/media:ro
    - certbot-www:/var/www/certbot:ro
    - letsencrypt:/etc/letsencrypt:ro
  depends_on: [web, asgi]

certbot:
  image: certbot/certbot
  volumes:
    - certbot-www:/var/www/certbot
    - letsencrypt:/etc/letsencrypt
  entrypoint: "/bin/sh -c 'trap exit TERM; while ;; do certbot renew; sleep 12h; d

volumes:
  pgdata:
  redisdata:
  staticfiles:
  mediafiles:
  certbot-www:
  letsencrypt:

```

This follows the well-established pattern of splitting concerns across Django+Gunicorn, Nginx reverse proxy, PostgreSQL, and Redis containers ([TestDriven Docker + Django; Saasitive Celery+Django+Redis](#)).

6.3 `.env` example

```
bash
```

```
# Django
```

```
DJANGO_SECRET_KEY=change-me-long-random
```

```
DJANGO_DEBUG=0
```

```
DJANGO_ALLOWED_HOSTS=rms.example.com,api.rms.example.com
```

```
# Database
```

```
POSTGRES_DB=rms
```

```
POSTGRES_USER=rms
```

```
POSTGRES_PASSWORD=strong-pw
```

```
DATABASE_URL=postgres://rms:strong-pw@db:5432/rms
```

```
# Redis
```

```
REDIS_URL=redis://redis:6379/0
```

```
CHANNEL_LAYERS_URL=redis://redis:6379/1
```

```
CELERY_BROKER_URL=redis://redis:6379/2
```

```
# Paymob (TEST keys in dev, LIVE in prod)
```

```
PAYMOB_PUBLIC_KEY=pk_live_xxx
```

```
PAYMOB_SECRET_KEY=sk_live_xxx
```

```
PAYMOB_HMAC=xxxxxxxxxxxxxxxxxxxx
```

```
PAYMOB_INTEGRATION_CARD=1234567
```

```
PAYMOB_INTEGRATION_WALLET=1234568
```

```
PAYMOB_INTEGRATION_KIOSK=1234569
```

```
PAYMOB_WEBHOOK_URL=https://api.rms.example.com/api/v1/payments/webhook/paymob/
```

```
PAYMOB_REDIRECT_URL=https://rms.example.com/payment-complete
```

```
# JWT
```

```
SIMPLE_JWT_SIGNING_KEY=different-from-django-secret
```

6.4 Nginx config (`nginx/conf.d/rms.conf`)

nginx

```
upstream django_wsgi { server web:8000; }
upstream django_asgi { server asgi:9000; }

server {
    listen 80;
    server_name rms.example.com api.rms.example.com;
    location /.well-known/acme-challenge/ { root /var/www/certbot; }
    location / { return 301 https://$host$request_uri; }
}

server {
    listen 443 ssl http2;
    server_name rms.example.com api.rms.example.com;
    ssl_certificate /etc/letsencrypt/live/rms.example.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/rms.example.com/privkey.pem;

    add_header Strict-Transport-Security "max-age=63072000; includeSubDomains" always;
    add_header X-Frame-Options DENY;
    add_header X-Content-Type-Options nosniff;

    client_max_body_size 10M;

    location /static/ { alias /var/www/static/; expires 30d; }
    location /media/ { alias /var/www/media/; expires 30d; }

    location /ws/ {
        proxy_pass http://django_asgi;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_read_timeout 86400;
    }

    location / {
        proxy_pass http://django_wsgi;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

6.5 Commands cheat sheet

```
bash
```

```
# first boot
```

```
docker compose build
```

```
docker compose up -d db redis
```

```
docker compose run --rm web python manage.py migrate
```

```
docker compose run --rm web python manage.py createsuperuser
```

```
docker compose up -d
```

```
# day to day
```

```
docker compose logs -f web
```

```
docker compose exec web python manage.py shell
```

```
docker compose exec db pg_dump -U rms rms | gzip > backup_$(date +%F).sql.gz
```

```
# SSL first-time
```

```
docker compose run --rm certbot certonly --webroot -w /var/www/certbot -d rms.example
```

```
docker compose restart nginx
```

7. Admin Dashboard UI (Django Templates + Bootstrap 5)

7.1 Recommended base template

Primary pick: CoreUI Free Bootstrap 5 Admin Template — MIT license, actively maintained (v5.x series, AI-friendly architecture), built on Bootstrap 5.3, no jQuery dependency, native dark/light modes ([CoreUI free template](#)).

Acceptable alternatives:

- **Tabler** (MIT, 12k+ stars, multi-framework) for a slightly more minimal aesthetic ([AdminLTE review 2026](#)).
- **AdminLTE 4** (MIT, 45k+ GH stars, Bootstrap 5.3 in v4) — battle-tested but aesthetically heavier ([AdminLTE](#)).
- **Volt** (Themesberg, MIT, no jQuery, vanilla JS) — very clean modern look ([Volt repo](#)).

Avoid SB Admin 2 — still on Bootstrap 4.

7.2 Pages to build

URL	Template	Purpose
<code>/dashboard/</code>	<code>dashboard/home.html</code>	KPIs, charts, live orders card
<code>/dashboard/menu/</code>	<code>menu/list.html</code>	DataTable with images, toggle-available
<code>/dashboard/menu/<id>/edit/</code>	form	crop images, drag-drop modifiers
<code>/dashboard/orders/</code>	—	Live list (HTMX-powered or WS-updated)
<code>/dashboard/orders/<uuid>/</code>	detail	Receipt preview, refund
<code>/dashboard/inventory/</code>	—	Low-stock flagged, quick purchase entry
<code>/dashboard/staff/</code>	—	Shift history, role assignment
<code>/dashboard/customers/</code>	—	LTV, orders, loyalty balance
<code>/dashboard/reports/</code>	—	Date-range picker, export CSV/PDF
<code>/dashboard/settings/</code>	—	Tax rate, currency, printer, Paymob keys
<code>/cashier/</code>	POS screen	Big touch buttons, quick add, payment modal
<code>/kds/</code>	KDS board	Card per order, "READY" button, WS-driven

7.3 Charts

Use **Chart.js 4** directly in templates (no need for `django-chartjs` — it's lightly maintained and Chart.js is trivial to embed). Feed data either via a small JSON DRF endpoint (`/api/v1/analytics/revenue/?range=7d`) consumed by `fetch()` in the template, or inline the data with `|json_script` for small datasets. For advanced visuals (heatmaps), use **ApexCharts** (MIT).

7.4 Making Bootstrap look non-generic

Ship a thin `static/css/rms.css` that overrides Bootstrap SCSS variables and adds a custom identity — this is the single highest-leverage move and avoids the classic "generic admin" trap:

SCSS

```
// 1. Own your palette (restaurant warm + trustworthy)
$primary: #E76F51; // warm paprika – used for primary actions
$secondary: #264653; // deep teal – used for navigation
$success: #2A9D8F;
$danger: #E63946;
$font-family-sans-serif: "Inter", "Cairo", system-ui, -apple-system, sans-serif; //
$border-radius: 0.625rem;
$box-shadow: 0 1px 2px rgba(15,23,42,.04), 0 4px 12px rgba(15,23,42,.06);
$card-border-width: 0;
@import "bootstrap/scss/bootstrap";
```

Additional polish techniques:

1. **Typography:** pair Inter (Latin) + Cairo (Arabic) — both on Google Fonts, designed to harmonize. Set `font-weight: 600` on headings, `letter-spacing: -0.01em`.
2. **Spacing:** adopt an 8-pt grid — use Bootstrap utilities `p-4`, `gap-4`, avoid the default `p-2` everywhere.
3. **Shadow hierarchy:** use `$box-shadow-sm` on hover, `$box-shadow` on cards, never `$box-shadow-lg` (feels dated).
4. **Microinteractions:** `transition: transform .15s ease; :hover { transform: translateY(-1px); }` on cards.
5. **Iconography:** use **Lucide** or **Tabler Icons** (both MIT, clean line style) rather than Font Awesome free.
6. **Empty states:** illustrate them (undraw.co) instead of grey text.
7. **Arabic/RTL:** enable `dir="rtl"` dynamically via middleware; Bootstrap 5 has native RTL support (`bootstrap.rtl.min.css`).

8. Statistics & Analytics for Admin

8.1 KPIs to compute

Category	KPI	Source
Revenue	Today / 7d / 30d / custom; YoY delta	<code>Sum(total) W</code> <code>payment_statu</code>
Orders	Count, cancellation %, refund %	<code>Count,</code> <code>Count(status</code> <code>Count(*)</code>
AOV (Average Order Value)	<code>Sum(total)/Count(id)</code>	Industry standar
Top-selling items	<code>OrderItem.values('menu_item').annotate(Sum('quantity'))</code>	Lightspeed KPI
Menu-item profitability	<code>(price - cost) * units_sold</code>	NetSuite restaur
Peak hours heatmap	<code>TruncHour(created_at)</code> grouped by weekday	Rezku POS analy
Payment- method breakdown	<code>Payment.values('method').annotate(Sum('amount'))</code>	—
Customer acquisition	New customers per day; repeat-customer rate	—
LTV	<code>User.annotate(ltv=Sum('orders__total'))</code>	—
Inventory turnover	<code>ingredient_sold / avg_stock</code>	NetSuite
Low-stock alerts	<code>Ingredient.filter(stock_qty <= low_threshold)</code>	—
Staff performance	Orders handled, avg order value, avg prep time per cashier/waiter	Lightspeed
Table turnover	<code>parties_served / table_count / open_hours</code>	Toast
RevPASH	<code>revenue / (seats × hours)</code>	MyPlace

8.2 Efficient Django ORM patterns

All aggregations should be pushed to the database via `aggregate()` / `annotate()`—never loop in Python ([Django aggregation docs](#); [Stanza practical guide](#)):

python

```
# analytics/services.py
from django.db.models import Sum, Count, Avg, F, Q
from django.db.models.functions import TruncDay, TruncHour, ExtractWeekDay
from django.core.cache import cache

def dashboard_overview(start, end):
    key = f"kpi:overview:{start}:{end}"
    if (cached := cache.get(key)):
        return cached

    paid = Order.objects.filter(
        payment_status=Order.PaymentStatus.PAID,
        created_at__range=(start, end),
    )
    stats = paid.aggregate(
        revenue=Sum("total"),
        orders=Count("id"),
        aov=Avg("total"),
    )
    stats["top_items"] = list(
        OrderItem.objects.filter(order__in=paid)
        .values("menu_item__name")
        .annotate(units=Sum("quantity"),
            revenue=Sum(F("unit_price") * F("quantity")))
        .order_by("-units")[:10]
    )
    stats["by_day"] = list(
        paid.annotate(day=TruncDay("created_at"))
        .values("day").annotate(total=Sum("total")).order_by("day")
    )
    stats["peak_hours"] = list(
        paid.annotate(hour=TruncHour("created_at"),
            weekday=ExtractWeekDay("created_at"))
        .values("hour", "weekday")
        .annotate(orders=Count("id")).order_by("weekday", "hour")
    )
    stats["payment_methods"] = list(
        Payment.objects.filter(order__in=paid, status=Payment.Status.SUCCESS)
        .values("method").annotate(total=Sum("amount"))
    )
    cache.set(key, stats, timeout=300) # 5-min TTL
    return stats
```

Caching layer via `django-redis` — cache expensive aggregations with a short TTL (5 minutes is usually safe for dashboards), and invalidate on paid-order creation via signals ([django-redis official](#); [OneUptime Django+Redis caching](#)). Be careful to **encode the date range in the cache key** — accidental cache collisions across users/ranges is the #1 cache-correctness bug ([dev.to Django+Redis pitfalls](#)).

8.3 Scaling analytics

When the orders table passes ~1M rows, migrate reporting queries to a **PostgreSQL materialized view** refreshed every 5 minutes via Celery Beat:

sql

```
CREATE MATERIALIZED VIEW mv_daily_revenue AS
SELECT DATE(created_at) AS day, SUM(total) AS revenue, COUNT(*) AS orders
FROM orders_order WHERE payment_status = 'paid' GROUP BY 1;
CREATE UNIQUE INDEX ON mv_daily_revenue(day);
```

python

```
@shared_task
def refresh_analytics_views():
    with connection.cursor() as c:
        c.execute("REFRESH MATERIALIZED VIEW CONCURRENTLY mv_daily_revenue;")
```

9. KDS Real-Time Implementation

9.1 ASGI wiring

`rms/asgi.py`:

python

```
import os
from django.core.asgi import get_asgi_application
from channels.routing import ProtocolTypeRouter, URLRouter
from channels.auth import AuthMiddlewareStack

os.environ.setdefault("DJANGO_SETTINGS_MODULE", "rms.settings.production")
django_asgi = get_asgi_application()

from orders.routing import websocket_urlpatterns

application = ProtocolTypeRouter({
    "http": django_asgi,
    "websocket": AuthMiddlewareStack(URLRouter(websocket_urlpatterns)),
})
```

settings.py channel layer:

python

```
CHANNEL_LAYERS = {
    "default": {
        "BACKEND": "channels_redis.core.RedisChannelLayer",
        "CONFIG": {"hosts": [os.environ["CHANNEL_LAYERS_URL"]]},
    },
}
```

9.2 Consumer (orders/consumers.py)

python

```
import json
from channels.generic.websocket import AsyncJsonWebSocketConsumer
from channels.db import database_sync_to_async
from accounts.models import Role

class KDSConsumer(AsyncJsonWebSocketConsumer):
    group_name = "kds"

    async def connect(self):
        user = self.scope["user"]
        if not user.is_authenticated or user.role not in {Role.KITCHEN, Role.MANAGER}:
            await self.close(code=4401)
            return
        await self.channel_layer.group_add(self.group_name, self.channel_name)
        await self.accept()
        await self.send_json({"event": "connected"})

    async def disconnect(self, code):
        await self.channel_layer.group_discard(self.group_name, self.channel_name)

    async def receive_json(self, content, **kwargs):
        if content.get("event") == "order.status":
            await self.update_status(content["order_id"], content["status"])

    @database_sync_to_async
    def update_status(self, order_id, status):
        from orders.models import Order
        Order.objects.filter(id=order_id).update(status=status)

    # handler for events pushed via group_send
    async def order_new(self, event):
        await self.send_json(event)
    async def order_updated(self, event):
        await self.send_json(event)
```

orders/routing.py):

python

```
from django.urls import re_path
from .consumers import KDSConsumer, OrderTrackConsumer

websocket_urlpatterns = [
    re_path(r"ws/kds/$", KDSConsumer.as_asgi()),
    re_path(r"ws/order/(?P<uuid>[0-9a-f-]+)/$", OrderTrackConsumer.as_asgi()),
]
```

9.3 Pushing events from services

python

```
# orders/services.py
from asgiref.sync import async_to_sync
from channels.layers import get_channel_layer

def on_order_paid(order):
    channel_layer = get_channel_layer()
    payload = {"type": "order.new", "event": "order.new",
               "order": OrderSerializer(order).data}
    async_to_sync(channel_layer.group_send)("kds", payload)
    async_to_sync(channel_layer.group_send)(f"order_{order.uuid}", {
        "type": "order.updated", "event": "order.updated",
        "status": order.status,
    })
```

9.4 KDS client recommendation

Use a web-based KDS rendered with Bootstrap 5 + vanilla JS over WebSocket — simpler to deploy on a tablet in the kitchen, works on any browser, no app-store hassle. Only build a Flutter KDS if you need offline resilience with local queueing. Sample JS:

html

```
<script>
const ws = new WebSocket(`wss://${location.host}/ws/kds/`);
ws.onmessage = (e) => {
    const data = JSON.parse(e.data);
    if (data.event === "order.new") renderOrderCard(data.order);
    if (data.event === "order.updated") updateCardStatus(data);
};
function markReady(orderId) {
    ws.send(JSON.stringify({event:"order.status", order_id:orderId, status:"ready"}));
}
</script>
```

10. Flutter App Architecture

10.1 State management

Riverpod 3 is the recommended default for 2026 new Flutter projects — compile-time safety, minimal boilerplate, and built-in support for async/offline state ([Foresight Mobile best state management 2026](#)). Bloc is the fallback choice if the developer is already fluent in it or if strict event-sourcing audit trails are needed ([Creole Studios comparison](#)).

10.2 Folder structure (feature-first + light clean architecture)

```
lib/
├─ main.dart
├─ app/
│   ├─ app.dart          # MaterialApp, routing (go_router)
│   ├─ theme.dart        # colors, typography (with Arabic fallback)
│   └─ router.dart
├─ core/
│   ├─ api/
│   │   ├─ dio_client.dart # interceptors: auth, refresh, logging
│   │   └─ api_exceptions.dart
│   ├─ storage/
│   │   └─ secure_storage.dart
│   ├─ constants.dart
│   └─ utils/
├─ features/
│   ├─ auth/
│   │   ├─ data/          # AuthRepository, models
│   │   ├─ domain/        # entities, usecases (optional)
│   │   └─ presentation/  # providers (riverpod), screens, widgets
│   ├─ menu/
│   ├─ cart/
│   ├─ checkout/
│   ├─ orders/
│   ├─ profile/
│   └─ loyalty/
└─ shared_widgets/
```

This mirrors the feature-first pattern recommended for scalable Flutter projects ([DEV.to feature-first vs clean](#)).

10.3 Key packages (pubspec.yaml)

yaml

```
dependencies:
  flutter_riverpod: ^3.0.0
  riverpod_annotation: ^3.0.0
  go_router: ^14.0.0
  dio: ^5.7.0
  flutter_secure_storage: ^9.2.2
  cached_network_image: ^3.4.0
  intl: ^0.19.0
  freezed_annotation: ^2.4.4
  json_annotation: ^4.9.0
  web_socket_channel: ^3.0.1
  paymob: ^latest # payments
  image_picker: ^1.1.2 # avatar
  permission_handler: ^11.3.1
  flutter_local_notifications: ^17.2.0
  connectivity_plus: ^6.0.5
  shimmer: ^3.0.0 # loading placeholders
  qr_code_scanner_plus: ^2.0.0 # scan table QR

dev_dependencies:
  build_runner: ^2.4.13
  riverpod_generator: ^2.4.0
  freezed: ^2.5.7
  json_serializable: ^6.8.0
```

10.4 Screens to implement

1. **Splash** — check for valid access/refresh token
2. **Onboarding / Language picker** (AR/EN)
3. **Auth** — login, register, forgot password, OTP
4. **Home / Menu** — category carousel, featured items, search
5. **Category listing** — filter by price/tags
6. **Item detail** — image gallery, modifier selection, qty stepper
7. **Cart** — review, apply coupon, select order type
8. **Checkout** — address picker (for delivery), order notes
9. **Payment** — method selector → Paymob WebView → status
10. **Order tracking** — live status via WebSocket (placed → preparing → ready → out for delivery)
11. **Order history** — paginated list
12. **Order detail / receipt** — reorder button
13. **Profile** — edit info, manage addresses
14. **Loyalty** — points balance, tier progress, redemption history
15. **Settings** — language, notifications, logout

10.5 JWT interceptor (Dio)

dart

```
class AuthInterceptor extends Interceptor {
  final FlutterSecureStorage _storage = const FlutterSecureStorage();
  @override
  Future<void> onRequest(o, h) async {
    final t = await _storage.read(key: 'access');
    if (t != null) o.headers['Authorization'] = 'Bearer $t';
    h.next(o);
  }
  @override
  Future<void> onError(e, h) async {
    if (e.response?.statusCode == 401) {
      if (await _refresh()) {
        // retry original request
        final clone = await Dio().fetch(e.requestOptions);
        return h.resolve(clone);
      }
    }
    h.next(e);
  }
}
```

11. Step-by-Step Build Order (Solo developer roadmap)

Realistic estimate for a solo full-stack engineer working ~20–25 productive hours/week:

Phase	Deliverables	Duration
Phase 1 — Foundation	Docker-compose up; Postgres+Redis+Django healthy; custom <code>User</code> model; JWT auth; CORS; base template; CI (GitHub Actions running tests + ruff + docker build)	Week 1–2
Phase 2 — Menu	Category, MenuItem, Modifier models + admin forms; image upload; DRF endpoints <code>/menu/*</code> + pagination; admin CRUD template	Week 3
Phase 3 — Orders (core)	Order, OrderItem, Table models; cart → order flow via POS UI & Flutter API; transitions & validation; invoice receipt template	Week 4–5
Phase 4 — Payment	Paymob sandbox integration; intention API; webhook HMAC verification; retries; failed-payment recovery; Flutter Paymob SDK flow	Week 6
Phase 5 — Flutter MVP	Project scaffold; Riverpod; auth; menu browsing; cart; checkout; payment; order tracking (polling first); secure storage	Week 7–9
Phase 6 — KDS real-time	Django Channels + Redis; KDS web screen; broadcast on paid; upgrade Flutter to WebSocket for order tracking	Week 10
Phase 7 — Inventory	Ingredient, Recipe, StockTransaction; atomic deduction on order paid (select_for_update); low-stock email/push	Week 11
Phase 8 — Analytics & Loyalty	Dashboard aggregations; Chart.js templates; Redis caching; loyalty points ledger on paid orders; customer tier display	Week 12–13
Phase 9 — Shift management, reports, polish	Shift open/close; cash drawer reconciliation; CSV/PDF export; Arabic RTL; a11y audit	Week 14
Phase 10 — Hardening & deploy	Rate limiting; security audit; load test (k6); SSL via Certbot; PostgreSQL backup job; monitoring (Sentry); staging→production migration	Week 15–16

Total: ~3.5–4 months for a polished v1. Add 2–3 weeks if unfamiliar with Django Channels or Flutter.

Skipping the Flutter app (web-only) would shorten this to ~9–10 weeks.

12. Security & Best Practices

12.1 Django REST API

- **Authentication:** `rest_framework_simplejwt` with short-lived access (15 min) + refresh rotation + blacklist on logout ([DRF auth guide](#)).
- **Permissions:** default `IsAuthenticated`; per-viewset `IsAdmin`, `IsCashier`, `IsOwnerOrAdmin` using DRF permission classes; keep role checks off `request.user.role` in a custom permission class.
- **Rate limiting:** DRF built-in throttling classes (`AnonRateThrottle`, `UserRateThrottle`), configured per view for auth endpoints (e.g., `5/min` on login), plus `django-ratelimit` for non-DRF views ([DRF throttling](#); [django-ratelimit security](#)). Back the throttle cache with Redis to make limits cluster-wide.
- **CORS:** `django-cors-headers` with an **explicit allow-list** of your Flutter app's domain and admin domain; **never** use `CORS_ALLOW_ALL_ORIGINS=True` in production ([StackHawk Django CORS guide](#); [Blueshoe CORS](#)).
- **Validation:** always pass through serializers; never trust `request.data`. Use `serializers.ValidationError` with `validators=[...]`.
- **SQL injection:** Django ORM parametrizes automatically; reserve `.raw()` for audited reports only.
- **Django security middleware:** `SecurityMiddleware` + `SECURE_SSL_REDIRECT=True`, `SECURE_HSTS_SECONDS=31536000`, `SECURE_HSTS_INCLUDE_SUBDOMAINS=True`, `SESSION_COOKIE_SECURE=True`, `CSRF_COOKIE_SECURE=True`, `X_FRAME_OPTIONS='DENY'`, `SECURE_CONTENT_TYPE_NOSNIFF=True`, `SECURE_REFERRER_POLICY='same-origin'`.
- **Secrets:** never commit; `.env` via `django-environ`; production secrets stored as Docker Swarm/Compose secrets or environment injected by the hosting platform.
- **Error handling:** integrate **Sentry** for both Django and Flutter — set `send_default_pii=False`, scrub headers.

12.2 Payment data protection

- **Never store PAN/CVV.** Paymob's Unified Checkout / iFrame means card data is submitted directly to Paymob — your server scope stays at PCI-DSS SAQ-A.
- **Use the hosted checkout** (or Paymob Pixel) rather than a raw card form, which would force SAQ-D (major additional audit burden).
- **Tokenization for recurring:** if you later want "save card" UX, use Paymob's saved-card tokens — you store the token, never the card ([Paymob saved card flow](#)).
- **Verify the webhook signature** on every callback with `hmac.compare_digest` (constant-time) — reject if mismatch; assume any unverified request is hostile.
- **Idempotency:** use `special_reference=order-<uuid>` so duplicate webhooks don't double-credit orders.

12.3 HTTPS and infrastructure

- **Let's Encrypt** via Certbot in a sidecar container with auto-renewal every 12h (see compose above).
- HSTS enabled on root + API domains (1 year).
- Redis/Postgres never exposed to the public network — only accessible within the docker network.
- Limit Nginx `client_max_body_size` to 10 MB.

12.4 Flutter app hardening

- **Certificate pinning** via Dio `badCertificateCallback` with known SHA-256 fingerprints of your API and Paymob's domains — rotate annually.
- **Obfuscation**: build with `--obfuscate --split-debug-info=./symbols`; ship only the obfuscated bundle to the store.
- **Secure storage**: `flutter_secure_storage` (Keystore on Android / Keychain on iOS) for refresh tokens; never `SharedPreferences` for secrets.
- **Root/jailbreak detection** (optional): use `freerasp` for high-value deployments.
- **Anti-tamper**: Play Integrity / App Attest hooks; consult Paymob documentation for their mobile-SDK integrity requirements.

12.5 PostgreSQL backup strategy

- **Daily full logical dump** via Celery Beat:

```
python

# rms/celery.py
from celery.schedules import crontab
app.conf.beat_schedule = {
    "nightly-db-backup": {"task": "ops.tasks.db_backup", "schedule": crontab(hour=2,
}
```

```
python

# ops/tasks.py
import subprocess, boto3, datetime
@shared_task
def db_backup():
    ts = datetime.datetime.utcnow().strftime("%Y%m%dT%H%M%S")
    file = f"/tmp/rms_{ts}.sql.gz"
    subprocess.check_call(f"pg_dump $DATABASE_URL | gzip > {file}", shell=True)
    s3 = boto3.client("s3", endpoint_url=settings.S3_ENDPOINT)
    s3.upload_file(file, settings.BACKUP_BUCKET, f"db/{ts}.sql.gz")
```

- **Offsite storage:** Backblaze B2 or Contabo S3 (cheap, available in EU regions close to Egypt). Retain 30 daily + 12 monthly backups.
 - **Test restores monthly** to a throwaway container — an untested backup is no backup.
 - **WAL archiving (optional, for ≤ 5 -min RPO):** enable `archive_mode=on` and stream WAL to S3 via `pgbackrest`.
-

13. Open Source Projects to Reference

These repos are worth studying or partially forking as you build:

1. `betofleitass/django_point_of_sale` — Django + Bootstrap POS with auth, products, sales, reporting. Good reference for POS UI patterns and Django template structure ([GitHub](#)).
2. `HarshilJ0/Restro_POS` — Restaurant POS in pure Django with categories, products, transactions, receipts, and sales reports; readable `views.py` you can learn from ([GitHub](#)).
3. `Epse/EpPos` — Compact Django POS already shipped with docker-compose; good starting scaffolding for your container setup ([GitHub](#)).
4. `dimitara/pypos` — Python + Django + Angular restaurant POS; useful for seeing how a split frontend/backend was structured pre-Flutter era ([GitHub](#)).
5. `omaarsaaleh/django-paymob-integration` — Minimal but well-documented Django + Paymob Accept/Intention + HMAC webhook sample — closest real reference for your payment flow ([GitHub](#)).
6. `PaymobAccept` [GitHub org](#) — official Paymob Python, Flutter, and JS SDK repositories; authoritative reference implementations ([GitHub Paymob org](#)).
7. `virajkothari7/OnlineRetailPOS` — Django POS with docker support and inventory; useful for the inventory module patterns ([GitHub](#)).
8. **URY (FOSS ERPNext restaurant)** — While built on the Frappe framework (not Django), URY is the most complete open restaurant management system (POS, table ordering, KOT) and worth studying for domain modeling decisions ([GitHub Topics — restaurant-management](#)).

Recommendation: Don't fork any of these wholesale — they're all simpler in scope than what you're building. Treat #5 (`omaarsaaleh/django-paymob-integration`) as a reference for the payment flow, and #1 (`django_point_of_sale`) for the admin template structure, then build your own from scratch. The domain model you need (inventory + modifiers + KDS + loyalty + Flutter-ready) is more sophisticated than any single OSS project, so a clean build is faster than reshaping an existing one.

- **Paymob pricing** (2.75% + 3 EGP) is the widely cited SME rate as of 2024–2025 from multiple third-party sources ([Paymob's own pricing page](#); [inai.io](#); [Hulkapps](#); [road9media](#)). Actual rate depends on monthly volume and MCC — **get a written quote** for a restaurant MCC before committing.
- **Stripe is not available for Egyptian merchants** without forming a foreign (typically US) LLC plus bank account — explicitly confirmed by multiple sources ([Rapyd Stripe countries](#); [OneSafe](#); [persuasion-nation](#)). Do not plan on it.
- **Paymob's Intention API** is the current-generation integration. Older Paymob tutorials referencing the three-call "auth → order → payment key → iframe" flow still work but are being deprecated in favor of the single Intention call + `client_secret` + Unified Checkout pattern ([Paymob docs](#)).
- **HMAC verification field order** occasionally changes; always re-verify with Paymob's latest developer docs at integration time ([Paymob developer portal](#)).
- **Hosting:** for Egyptian customers, low-latency hosting is available from Contabo (Germany), Hetzner (Germany/Finland), and AWS Bahrain / Frankfurt. Local hosting options like TE Data Cloud exist but review Paymob/CBE data residency expectations if storing PII.
- **VAT:** Egypt's 14% VAT applies to restaurant sales above the registration threshold; the `Order.tax` field and tax-settings page should default to 14% for Egypt and be admin-configurable.
- Some "top 10 payment gateways in Egypt" listicles (e.g., [ThruHQ](#), [NAOS Solutions](#), [TheFinrate](#)) are commercial/affiliate content and their rankings should be cross-checked against vendor documentation before relying on any specific claim beyond availability and general feature set.

This blueprint is opinionated where the evidence is strong (Paymob, Channels, Riverpod, PostgreSQL + JSONB, CoreUI) and flexible where team preference legitimately matters (Bloc vs Riverpod, CoreUI vs Tabler). Build Phase 1 end-to-end before optimizing anything — ship a thin vertical slice (register → browse menu → place cash order → see it in KDS) in the first 3 weeks and iterate from real usage feedback.